5.6. Extending built-ins

You've seen that classes in CoffeeScript are syntactic sugar. They make working with constructors and prototypes easier. There are constructors for objects other than classes you define. Consider an array, an object, and a string:

```
object = {}
array = []
string = ''
```

Objects, arrays, and strings each have their own constructor functions. Instead of using the literal notation, it's possible, if generally pointless, to create objects, arrays, and strings using constructors:

```
object = new Object
array = new Array
string = new String
```

The prototypes of these objects provide the methods available to all objects, arrays, and strings. In a sense, there's a built-in `Object` class, an `Array` class, and a `String` class. But you now know that classes are really a syntactic convenience in CoffeeScript, so it's more accurate to talk of built-in prototypes and constructor functions.

By modifying built-in prototypes, you can change the behavior of the built-in language. This is a powerful and dangerous technique. In this section you'll learn how you can use it and why it should be treated with caution.

5.6.1. Built-in constructor prototypes

The built-in constructors provided to you will depend on the runtime you're using. The constructors you're most likely to encounter are `Object`, `Array`, `String`, `Date`, and `RegExp`.[1] All of these built-in constructors have prototypes that you can modify. For example, consider the `join` method on an array:

[1] A full list of the standard constructors specified by ECMAScript 5 is included in appendix A.